

Exact Census and Simultaneous Shannon Size–Depth Bounds for Conservative Terms over a Fixed Three-Element Algebra

Dana Edwards

Review draft — 13 August 2026

Abstract

Fix the three-element algebra

$$Q = (\{0, 1, 2\}; d, u),$$

where $d(x, y, z) = z$ when $x = y$ and $d(x, y, z) = x$ otherwise, and where $u(0) = 1$, $u(1) = 0$, and $u(2) = 1$. We study the parameter-free r -ary term operations of Q that are conservative. Their number is exactly

$$|\text{CT}_r(Q)| = 2^{5 \cdot 2^{r-1} - 5} \cdot 3^{3^r - 3 \cdot 2^r + 3}.$$

For an operation f , let $C_r(f)$ be the minimum number of original-signature operation nodes in a rooted term DAG computing f , with free fan-out, and let $D_r(f)$ be the corresponding minimum operation depth. We give a parameter-free Lupanov-style compiler and prove

$$\max_{f \in \text{CT}_r(Q)} C_r(f) = \Theta(3^r/r).$$

The lower bound holds for almost every uniformly selected member of $\text{CT}_r(Q)$. A sequence of depth refinements includes an exact recursive signed-router family and an exact sibling-shared program-vector construction. For $q = 3^w$, either fixed signed-router vector has local size $S(w)$ and depth $D(w)$ satisfying

$$3 \cdot S(w) \leq 4q + 15 \cdot 3^{\lceil w/2 \rceil}$$

and

$$D(w) \leq 3 + \lceil \log_2 w \rceil.$$

Its leading size $(4/3 + o(1))q$ matches a $4q/3$ scalar-output lower bound in the declared address-only model. This is an exact local theorem, not a global compiler lower bound. An explicit two-plane construction and a complement-relative binary construction culminate in one and the same DAG having

$$\text{size} = O(3^r/r)$$

and

$$\text{depth} \leq r + O(\log r).$$

More explicitly, for $r \geq 64$ the construction has size below $34 \cdot 3^r/r$ and depth at most $r + 4 \cdot \lceil \log_2 r \rceil + 9$.

Writing $\Delta_r(Q) = \max_{f \in \text{CT}_r(Q)} D_r(f)$, syntax counting also gives the almost-all lower bound

$$D_r(f) \geq r - \log_3(\log r) - O(1),$$

and therefore the worst-case interval

$$r - \log_3(\log r) - O(1) \leq \Delta_r(Q) \leq r + O(\log r).$$

The displayed upper bound is a manuscript theorem with an explicit generic proof and a separate no-author-import executable reconstruction. It remains unformalized as an integrated Lean DAG theorem and does not settle the optimal additive term. The independently audited growing-router schedule supplied the earlier $r + O(\sqrt{r})$ stage after a required residual-first repair. Historical R6/R7 and R9 bounds remain visible as prior compiler stages. The construction combines an exact conservative-clone census, dynamic value names, classical local coding, recursive subcube libraries, a balanced absorbing anchor, signed discriminator routers, and

parallel address programs. Deterministic experiments check finite semantics, compiler schedules, mutations, and structural arithmetic. They are validation evidence, not substitutes for the generic proofs. No publication novelty, legal conclusion, practical performance, or relationship to private Tau work is claimed.

Contents

1. Introduction	3
2. Algebra, semantic class, and cost model	4
3. Semantic characterization and exact enumeration	5
4. Counting lower bounds	6
5. Parameter-free local coding	7
6. Simultaneous size and depth	9
7. Compiler progression and bounded validation	21
8. Relation to prior work	25
9. Provenance, Tau boundary, and explicit nonclaims	27
10. Remaining and quarantined work	28
11. Reproducibility map	28
Declarations	31
References	31

Review and authorship status. This PDF is a non-archival review copy. The author is Dana Edwards. Affiliation, corresponding-author contact, ORCID, contribution statement, funding, competing interests, and final archival approval remain pending. Do not deposit this file until those gates are closed. The mathematical source is bound to SHA-256 01ddf4565212f798d0adf5c4c9b16d6de9074a317a34f2a39a80051de55a5410 and repository commit [ab20b9df50f3](#). The manuscript has not been externally peer reviewed. Novelty, patent freedom to operate, license interpretation, and practical performance remain unknown and are not claimed.

Keywords: finite algebra; conservative operation; discriminator; term DAG; Shannon complexity; local coding; multivalued logic; circuit depth.

MSC 2020: 08A70 (applications of universal algebra in computer science); 68Q06 (networks and circuits as models of computation; circuit complexity).

Claim status at a glance

Object	Status in this draft	What is not claimed
Exact census, given the stated Pixley specialization	unconditional manuscript theorem	new discriminator interpolation theory
$L_r(Q) = \Theta(3^r/r)$ for shared $\{d, u\}$ DAGs	unconditional manuscript theorem	an exact leading size constant or an ordinary-tree theorem
$\Delta_r(Q) = \Theta(r)$ and leading depth coefficient one	unconditional manuscript theorem	an optimal additive-depth term

Object	Status in this draft	What is not claimed
d-only signed family P_h, N_h	unconditional exact local theorem	an optimal router in every grammar
sibling-shared program vector, Theorem 6.9	unconditional exact local theorem in its declared $A = 2$, scalar-output, free-fanout model	simultaneous P/N cost, formula cost, or an integrated compiler lower bound
$O(3^r/r)$ size together with $r + O(\log r)$ depth	unconditional manuscript theorem; independently reconstructed, not end-to-end Lean-formalized	an optimal additive-depth theorem, formula bound, or bounded-fanout bound
novelty, FTO, patents, Tau/license relationship, practical speed	UNKNOWN / not claimed	legal or commercial clearance

1. Introduction

Shannon complexity asks how large a circuit must be in the worst case, and typically for almost every function, when the function class and basis are fixed. Classical work treats Boolean circuits,
70 k -valued functional constructions, functionally complete bases, and local-coding synthesis. The object here differs in two linked respects:

1. the original signature $\{d, u\}$ is parameter-free and preserves the proper subalgebra $\{0, 1\}$, so it is not functionally complete on Q ; and
2. the semantic class is the conservative part of the term clone of one fixed algebra.

75 The first issue prevents us from simply naming constants. The second makes the semantic entropy slightly smaller than that of all ternary functions while retaining the leading term $(\log 3)3^r$. The parameter-free obstacle is handled constructively: a nonbinary input supplies dynamic names for 0, 1, 2, first locally and later through one balanced global anchor.

The contribution stack established in this draft is:

- 80 1. an exact closed-form census of the conservative r -ary term operations;
2. a complete coordinate-selector representation of that fragment;
3. worst-case and almost-all operation-node lower bounds of order $3^r/r$;
4. a matching parameter-free shared-DAG compiler obtained by specializing classical Lupanov local coding;
- 85 5. worst-case and almost-all depth lower bounds of order r ;
6. an explicit compiler progression from exponential Gray dependency depth to linear depth, then to $4r$, $3r + O(\log r)$, the historical R9 bound $(3/2)r + O(r/\log r)$, the signed-router bound $r + O(\sqrt{r})$, and finally the integrated parallel-program bound $r + O(\log r)$, always preserving $O(3^r/r)$ size on the same DAG;
- 90 7. exact six-way ternary, seven-way Boolean, and nine-way Boolean projection gadgets, retained as historical stages;
8. an exact d-only signed family P_h, N_h with 3^{h-1} programmable branches at dependency depth h , independently audited and Lean-checked;

9. an exact sibling-shared absorbing-rail address summary that, for either fixed sign, materializes all controls for a capacity- q signed router in $(4/3 + o(1))q$ shared-DAG nodes and $O(\log \log q)$ preprocessing depth, with a matching leading scalar-output lower bound in the declared local model; and
10. source-auditable bounded checks with mutation controls, replay hashes, two independent program-vector specifications, and an independent reconstruction of the integrated compiler.
- The local-coding principle, recursive Shannon expansion, discriminator interpolation, and general Shannon-counting methodology are not contributions of this work. The paper-worthy candidate, subject to full prior-art review, is their exact specialization and simultaneous realization for this fixed parameter-free incomplete clone.

2. Algebra, semantic class, and cost model

2.1 The fixed algebra

Let

$$Q = (\{0, 1, 2\}; d, u)$$

with

$$\begin{aligned} d(x, y, z) &= z, & \text{if } x=y, \\ & x, & \text{otherwise,} \\ u(0) &= 1, & u(1) = 0, & u(2) = 1. \end{aligned}$$

- The binary set $B = \{0, 1\}$ is a proper subalgebra. Its nontrivial automorphism is complement, written $\bar{0} = 1$ and $\bar{1} = 0$. Notice that u is not conservative because $u(2) = 1$.

An r -ary operation $f : Q^r \rightarrow Q$ is **conservative** if

$$f(x_0, \dots, x_{r-1}) \in \{x_0, \dots, x_{r-1}\}$$

- for every input tuple. Let $CT_r(Q)$ be the set of conservative r -ary operations induced by parameter-free terms over $\{d, u\}$.

2.2 Shared term DAGs

A rooted original-signature term DAG has:

- r designated input nodes $x_0, \dots, x_{r-1}$;
- operation nodes labelled by unary u or ternary d ;
- ordered incoming edges matching the operation arity; and
- one designated output node.

Fan-out is free. Input nodes are not charged. For $f \in CT_r(Q)$, define:

- $C_r(f)$: the minimum number of operation nodes in such a DAG computing f ;
- $D_r(f)$: the minimum operation depth, with every input at depth zero and every operation one plus the maximum depth of its arguments;
- $L_r(Q) = \max_{f \in CT_r(Q)} C_r(f)$; and
- $\Delta_r(Q) = \max_{f \in CT_r(Q)} D_r(f)$.

The executable compiler reports all distinct reachable nodes, including the r inputs, and puts inputs at depth one. For any fixed emitted DAG, its total node count is its operation-node count plus
 135 at most r , and its reported depth is its analytic operation depth plus one. The emitted values
 therefore upper bound, rather than equal, the minima $C_r(f)$ and $D_r(f)$. All asymptotic theorems
 and displayed analytic depth bounds use the operation-node convention.

Ordinary unshared term trees are not the main cost model. The baseline compiler also has an
 expanded-tree analysis, but no order-optimal ordinary-tree theorem is claimed here.

140 3. Semantic characterization and exact enumeration

3.1 Pixley specialization

Pixley's internal-isomorphism characterization of quasi-primal term operations [1,2] implies the
 following specialization for the fixed algebra.

Proposition 3.1 (conservative term criterion). For $r \geq 2$, a conservative operation $f : Q^r \rightarrow Q$
 145 belongs to $\text{CT}_r(Q)$ exactly when its restriction to the binary cube commutes with simultaneous
 complement:

$$f(\overline{x_0}, \dots, \overline{x_{r-1}}) = \overline{f(x_0, \dots, x_{r-1})}$$

for every $x \in B^r$.

Proof. We spell out the internal-isomorphism catalog needed from Pixley's criterion. The only
 150 nonempty proper subalgebra is $B = \{0, 1\}$: it is closed under d and u ; a set containing 2 and closed
 under u also contains 1 and then 0, so it generates Q ; and neither singleton is closed under u .
 Because d is the discriminator, every permutation preserves d . On B , the permutations preserving
 u are the identity and complement. On Q , the identity is the only permutation preserving u : the
 unique element 2 outside the two-cycle $\{0, 1\}$ must be fixed, after which complement would send
 155 $u(2) = 1$ to 0 while $u(2)$ remains 1. There is no isomorphism between B and Q because their
 cardinalities differ.

Thus the only nonidentity internal isomorphism that can constrain a term operation is simultane-
 ous complement on B^r . Pixley's characterization says that an operation preserving the relevant
 subalgebras and internal isomorphisms is a term operation. Conservativity supplies preservation
 160 of each tuple's value set and hence of both B and Q ; on the binary cube the remaining condition
 is exactly the displayed equivariance. Conversely every term operation preserves subalgebras and
 commutes with their automorphisms, so the condition is necessary. For $r = 1$, conservativity forces
 the identity projection. This is a concrete application of classical theory, not a new interpolation
 theorem. $\square$

165 3.2 Exact census

Theorem 3.2 (exact conservative term count). For every $r \geq 1$,

$$|\text{CT}_r(Q)| = 2^{5 \cdot 2^{r-1} - 5} \cdot 3^{3^r - 3 \cdot 2^r + 3}.$$

Proof. Partition Q^r by the set of values appearing in the tuple.

On the binary cube, the 2^r tuples form 2^{r-1} complement pairs. The constant pair $\{0^r, 1^r\}$ is forced
 170 by conservativity and equivariance. Every other pair contains both values; choosing the value on
 one representative determines the complementary value on its partner. This contributes

$$2^{2^{r-1}-1}.$$

A nonconstant tuple over $\{0, 2\}$ has two conservative output choices, as does a nonconstant tuple over $\{1, 2\}$. Each alphabet supplies $2^r - 2$ such tuples. These independent choices contribute

$$2^{2 \cdot (2^r - 2)}.$$

The all-2 tuple is forced. Inclusion–exclusion gives

$$3^r - 3 \cdot 2^r + 3$$

tuples using all three values, and each has three conservative output choices. Their contribution is $3^{3^r - 3 \cdot 2^r + 3}$.

Multiplication and collection of the powers of two give the formula. The $r = 1$ instance equals one. $\square$

The first values are

$$\begin{aligned} |\text{CT}_1(Q)| &= 1, \\ |\text{CT}_2(Q)| &= 32, \\ |\text{CT}_3(Q)| &= 23,887,872. \end{aligned}$$

In particular,

$$\log |\text{CT}_r(Q)| = (\log 3)3^r + O(2^r).$$

3.3 Coordinate-selector completeness

Proposition 3.3. Every member of $\text{CT}_r(Q)$ has a complement-invariant coordinate-selector table.

Proof. For every tuple x , choose an index $j(x)$ such that $f(x) = x_{j(x)}$. On each binary complement pair choose the index on one representative and reuse it on its complement. Proposition 3.1 guarantees that the reused index produces the required complementary value. Nonbinary internal-isomorphism orbits are singletons. Conversely, the compilers below realize every such compatible selector table. $\square$

Selector tables need not be unique: a tuple can contain the desired output in several coordinates. The statement is semantic completeness, not a canonical encoding claim.

4. Counting lower bounds

4.1 Shared-DAG size

Theorem 4.1 (worst-case and almost-all size lower bound). There is a constant $c > 0$ such that

$$C_r(f) \geq c \cdot 3^r / r$$

for a fraction tending to one of the operations $f \in \text{CT}_r(Q)$. Consequently,

$$L_r(Q) = \Omega(3^r / r).$$

Proof. Topologically order a term DAG with s operation nodes. At operation position i , the number of choices for a unary node and its predecessor is at most $r + i$; the number for a ternary node and

its ordered predecessors is at most $(r + i)^3$. Choosing the output and summing over all sizes at most s changes only constants, so the number of descriptions is

$$\exp(O(s \cdot \log(r + s))).$$

205 For $s = c \cdot 3^r / r$, the exponent is $O(c \cdot 3^r)$. Theorem 3.2 gives $\exp((\log 3 + o(1)) \cdot 3^r)$ semantic operations. A sufficiently small fixed c leaves an exponentially vanishing fraction covered by the short descriptions. $\square$

This is a standard Shannon counting argument. It neither gives an optimal leading constant nor an $\Omega(3^r)$ lower bound.

210 4.2 Operation depth

Let $A_h(r)$ overcount the number of original-signature term trees of operation depth at most h on r variables. Unsharing a DAG does not increase depth, and

$$\begin{aligned} A_0(r) &= r, \\ A_{h+1}(r) &\leq r + A_h(r) + A_h(r)^3 \leq 3 \cdot A_h(r)^3 \end{aligned}$$

for $r \geq 2$. Therefore

$$\log A_h(r) \leq 3^h \cdot (\log r + (\log 3)/2).$$

215 **Theorem 4.2 (almost-all depth lower bound).** For a uniformly selected $f \in \text{CT}_r(Q)$,

$$D_r(f) \geq r - \log_3(\log r) - O(1)$$

with probability tending to one. In particular,

$$\Delta_r(Q) = \Omega(r).$$

220 **Proof.** If $h \leq r - \log_3(\log r) - K$ for a sufficiently large fixed K , the logarithm of the number of depth- h syntactic candidates is a sufficiently small constant multiple of 3^r , whereas Theorem 3.2 gives semantic log-cardinality $(\log 3 + o(1))3^r$. The fraction representable at that depth tends to zero exponentially in 3^r . $\square$

The depth lower bound does not assume a bound on DAG size.

5. Parameter-free local coding

225 5.1 First-2 slices and dynamic value names

Partition the nonbinary tuples by the least coordinate a at which $x_a = 2$. On this slice,

230 $x_0, \dots, x_{(a-1)}$ are binary,
 $x_a = 2$,
 $x_{(a+1)}, \dots, x_{(r-1)}$ are unrestricted.

The slice has

$$N_a = 2^a \cdot 3^{r-a-1}$$

rows. Within it, the terms

235 $u(u(x_a)), \quad u(x_a), \quad x_a$

name 0, 1, 2, respectively. These are branch-local dynamic names, not parameter-free constant terms on all of Q^r .

240 The reachable first-2 slices exactly partition the $3^r - 2^r$ nonbinary tuples because

$$\sum_{a=0}^{r-1} 2^a \cdot 3^{r-a-1} = 3^r - 2^r.$$

This already yields a no-wasted-leaf classifier. In the executable's total-node convention, writing T_r^{base} for its emitted node count, it gives

$$T_1^{\text{base}} = 6, \\ T_r^{\text{base}} = 3 \cdot 3^r - 3 \cdot 2^{r-1} + 6r - 4 < 4 \cdot 3^r \text{ for } r \geq 2.$$

The corresponding fully expanded emitted term has recorded syntax size

245 $(35 \cdot 7^r - 9 \cdot 3^r - 50)/12$

and depth $4r + 1$ in the executable convention. These are properties of that compiler, not exact minima.

The constant-size equality selector used below is explicit:

$$\text{EqSel}(x, y; p, q) = d(d(x, y, p), d(x, y, q), q).$$

If $x = y$, the outer discriminator returns p ; if $x \neq y$, both inner discriminators return x and the outer one returns q . Thus EqSel has three operation nodes and two dependency levels.

250

5.2 Mixed-domain local-coding lemma

Lemma 5.1 (mixed binary/ternary local coding). Let

$$D = A_1 x \dots x A_n,$$

where each $|A_i|$ is two or three, and set $N = |D|$. In a circuit language with names for 0, 1, 2 and the displayed EqSel, every table $F : D \rightarrow Q$ has a shared circuit of size $O(N/n)$, uniformly over the mixture of coordinate alphabets.

255

Construction. For sufficiently large n , put $H = \lfloor \log_3(N/n) \rfloor$; the finitely many smaller cases are absorbed into the uniform constant. Choose a block of coordinates whose number M of assignments is maximal subject to $M \leq H$. Maximality gives $H/3 < M \leq H$, while $N \geq 2^n$ implies $M = \Theta(n)$. Only $O(\log M)$ block coordinates are needed.

260

Precompute the point indicators for the M block assignments. There are 3^M functions from these assignments to Q . A reflected ternary Gray order changes one table entry at a time, so one constant-size conditional update creates each successive library member from the preceding one. Thus the shared library costs $O(3^M)$, and

265 $3^M \leq 3^H \leq N/n.$

The remaining coordinates have $N/M = O(N/n)$ assignments. A mixed-radix decision tree classifies them and lets each leaf point into the shared block-function library. Point indicators, the library, and the prefix tree together cost $O(N/n)$. $\square$

270 This is a concrete specialization of Lupanov's classical local-coding principle [6], not a new general synthesis method.

5.3 Size-optimal parameter-free compilation

Apply Lemma 5.1 separately on every first-2 slice, using the slice-local names $u(u(x_a)), u(x_a), x_a$ from Section 5.1. No global anchor has been constructed or needed at this stage. On slice a , apply the lemma to its $n = r - 1$ varying coordinates; the $r = 1$ identity is the separate finite base case.
275 Uniformly absorbing the finitely many small n , the total nonbinary cost is

$$\sum_{a=0}^{r-1} O(N_a/r) = O(3^r/r).$$

The all-binary branch is classified modulo complement in $O(2^r)$ nodes, and the first-2 dispatcher costs $O(r)$. Both are lower order. Together with Theorem 4.1 this proves:

Theorem 5.2 (Shannon complexity).

280 $L_r(Q) = \Theta(3^r/r).$

Moreover, for some constant $c > 0$, a uniformly selected member of $CT_r(Q)$ has complexity at least $c \cdot 3^r/r$ with probability tending to one, while the compiler supplies the matching order for every member.

285 The first executable realization uses a Gray-scheduled library. Its size has the desired order, but the Hamiltonian dependency chain gives poor depth. The next sections remove that scheduling artifact.

6. Simultaneous size and depth

6.1 Layered prefix library

290 Order the M block assignments as $y_1, \dots, y_M$, with disjoint point indicators $chi_1, \dots, chi_M$. For every word $w \in Q^j$, build g_w so that it equals w_i at y_i for $i \leq j$ and equals zero at all unprocessed points. The zero extension reuses its parent; the one and two extensions each add one selector. Hence the exact selector census is

$$2 \cdot \sum_{j=0}^{M-1} 3^j = 3^M - 1,$$

295 the same as for the Gray library, but the dependency depth is only M selector levels. Combining this library with the prefix decision tree proves that one DAG can simultaneously have

$$\text{size} = O(3^r/r) \text{ and } \text{depth} = O(r).$$

Together with Theorem 4.2, this already establishes $\Delta_r(Q) = \Theta(r)$.

6.2 Recursive subcube library and the $4r$ bound

300 The layered construction is sharpened by recursively indexing the library by its actual mixed-radix block coordinates. If their alphabet sizes are $q_1, \dots, q_b \in \{2, 3\}$, let

$$M_j = \prod_{i=j}^b q_i.$$

At a binary level, one EqSel combines each pair of suffix functions. At a ternary level, two nested EqSel terms combine each triple. The exact number of selector calls is

$$S(q_1, \dots, q_b) = \sum_{j=1}^b (q_j - 1) \cdot 3^{M_j} < 3 \cdot 3^M.$$

305 The top level dominates because $M_{j+1} \leq M_j/2$. The library remains $O(3^M)$, while a full mixed-radix prefix tree with P leaves uses exactly $P - 1$ weighted selector calls.

The explicit depth accounting on first-2 slice a is

$$2 + 2a + 4(r - a - 1) + 2(a + 1) = 4r.$$

310 The four terms are, respectively, the deepest slice-local dynamic name, binary and ternary coordinate routing inside the slice, and the first-2 dispatcher. The all-binary branch has depth at most $4r - 2$.

Theorem 6.1 (same-DAG $4r$ construction). Every $f \in \text{CT}_r(Q)$ has one parameter-free original-signature DAG with

size $= O(3^r/r)$ and depth $\leq 4r$.

315 6.3 One global nonbinary anchor

Define

$$h(x, y) = d(x, u(u(x)), d(y, u(x), x)).$$

Lemma 6.2. For all $x, y \in Q$, $h(x, y) = 2$ exactly when $x = 2$ or $y = 2$; otherwise $h(x, y) = x$.

320 Use an ordered balanced binary tree whose leaves, from left to right, are $x_0, \dots, x_{r-1}$, and label every internal node by h ; call its root A . The leaf order is part of the construction. On the binary cube each h returns its left argument, so the tree returns its distinguished leftmost leaf x_0 ; an arbitrary reassociation with x_0 elsewhere is not asserted. Then

$$A = 2 \text{ exactly on nonbinary tuples,}$$

$$A = x_0 \text{ on the binary cube,}$$

$$\text{size}(A) = O(r),$$

$$\text{depth}(A) \leq 3 \cdot \lceil \log_2 r \rceil.$$

325 On the nonbinary branch, $u(u(A)), u(A), A$ are global dynamic names for 0, 1, 2. On the binary branch they deliberately are not global constants. This is what keeps the construction parameter-free and compatible with the preserved subalgebra B .

An exhaustive closure of binary-arity term functions through operation depth three has cumulative sizes 2, 4, 20, 270; the absorber's target semantics first occurs at depth three. This finite fact concerns this absorber, not the optimal depth of the final global compiler.

6.4 A branch-depth-three ternary router

330 On a branch where terms named 0, 1, 2 have the indicated values, define

```

mux(x, b0, b1, b2)
= d(
  d(0, d(x, 2, b0), d(0, x, b0)),
  0,
  d(d(1, x, b1), 1, d(x, 2, b2))
).

```

Lemma 6.3. For every $x, b0, b1, b2 \in Q$,

$$mux(x, b0, b1, b2) = b_x.$$

Every branch argument lies exactly three discriminator nodes below the root. Direct substitution for $x = 0, 1, 2$ proves the identity. With $0 = u(u(A))$, $1 = u(A)$, and $2 = A$, the displayed shared materialization has seven d nodes and the two shared u name nodes. It therefore has nine operation nodes, or fourteen total nodes if its five distinct raw terminals are also counted. Its operation depth is five relative to an input anchor. An older executable reported fifteen because it constructed $u(A)$ twice; that is an emitter artifact, not the shared-DAG census used here.

A finite Z3 QF_BV search reports UNSAT for branch depth at most two in a deliberately generous selector-only grammar: all 27 unary functions of the selector are admitted as free terminals; $b0, b1, b2$ are branch terminals; internal nodes are u or d ; and all 81 valuations are imposed on the complete depth-two syntax tree. Together with the displayed term, this makes depth three optimal only for that finite branch-scoped grammar. It is not a global depth lower bound and currently has no separately checked proof certificate.

6.5 Global library and final depth theorem

Put

$$H = \lfloor \log_3(3^r/r) \rfloor.$$

Choose b ternary coordinates so that their number of assignments $M = 3^b$ is maximal subject to $M \leq H$. Apart from finitely many small arities, $H/3 < M \leq H$, and therefore

$$\begin{aligned}
M &= \Theta(r), \\
3^M &\leq 3^r/r, \\
3^r/M &= O(3^r/r).
\end{aligned}$$

Build every Q -valued function on the local block recursively, using the router of Lemma 6.3 at each coordinate. The library costs $O(3^M)$. Route the remaining prefix coordinates with the same router; the prefix tree costs $O(3^r/M)$. Add the $O(r)$ global anchor and the lower-order $O(2^r)$ complement-orbit classifier for the binary branch. A final EqSel chooses that binary classifier exactly when $u(u(A)) = A$ and otherwise chooses the global nonbinary library.

The block and prefix partition the r routed coordinates. Each routed coordinate adds three branch-dependency levels. Put $\alpha = 3 \cdot \lceil \log_2 r \rceil$. The deepest dynamic name has depth $\alpha + 2$, and the nonbinary routed output before the final choice has depth at most $3r + \alpha + 2$. The binary complement-orbit classifier uses at most $r - 1$ sequential equality decisions, each one EqSel of branch-dependency depth two, so its depth is at most $2(r - 1)$. The final EqSel adds two levels. Therefore the second compiler has depth

```

370 2+max(alpha+2, 2(r-1), 3r+alpha+2)
    =3r+3*ceil(log_2 r)+4.

```

Taking the better of this construction and Theorem 6.1 gives:

Theorem 6.4 (simultaneous Shannon size and explicit depth). Every $f \in \text{CT}_r(Q)$ has one
375 parameter-free original-signature term DAG with

$$\text{size} = O(3^r / r)$$

and

$$\text{depth} \leq \min(4r, 3r + 3 \cdot \lceil \log_2 r \rceil + 4).$$

Combining Theorems 4.2 and 6.4 gives

$$370 \quad r - \log_3(\log r) - O(1) \leq \Delta_r(Q) \leq \min(4r, 3r + 3 \cdot \lceil \log_2 r \rceil + 4),$$

and in particular

$$r - \log_3(\log r) - O(1) \leq \Delta_r(Q) \leq 3r + O(\log r).$$

Thus this historical compiler already proves $\Delta_r(Q) = \Theta(r)$ with leading-coefficient interval $[1, 3]$.
The historical fixed-router stages below strictly narrowed that upper endpoint before the signed-
385 family construction. Neither endpoint is claimed optimal.

6.6–6.7 Historical fixed-router stages

Two fixed depth-three routers preceded the recursive signed family. They are retained for provenance and comparison, not because the optimized theorem depends on them.

- R6 has 13 discriminator nodes and six ternary branches. Its six fixed 18-slot programs realize
390 all branch projections.
- R7 has 12 discriminator nodes and seven Boolean branches. Its programs use the complement-relative binary names $x_0, u(x_0)$, not absolute constants.
- R9 is a full 13-node depth-three discriminator tree with nine Boolean branches and 18 program slots. Nine programs realize all nine projections.

395 Balanced six-way routing gave the historical nonbinary ceiling

$$(3 \cdot \log_6(3))r + O(r / \log r) = 1.839441578296 \dots r + O(r / \log r).$$

For R9, encode the dynamic values by $0 \rightarrow 00, 1 \rightarrow 01, 2 \rightarrow 10$, run two copies in parallel, and decode once with

$$\text{Dec}(\text{high}, \text{low}) = d(d(\text{high}, \text{one}, \text{two}), \text{zero}, \text{low}).$$

400 This gave the later historical ceiling

$$(3/2)r + O(r / \log r).$$

At every route level one shared address-control family serves all active routers (and both R9 planes), so the fixed-router schedules preserve $O(3^r / r)$ size on the same DAG. The exact telescoping router census, balanced-library recurrences, residual schedules, and all fixed programs are in `notes/QUASIPRIMAL_CONSERVATIVE_TERM_PROGRAMMABLE_DEPTH.md` and
405

notes/QUASIPRIMAL_CONSERVATIVE_TERM_BOOLEAN_R9_DEPTH.md. Deterministic replay covers 4,374 R6 cases, 896 R7 cases, 4,608 R9 projection cases, 9,216 complement-relative cases, and 177,147 two-plane cases. These bounded checks validate the gadgets; the all-arity schedule statements remain manuscript proofs and historical calibration.

410 The old bounds were, respectively,

$$\begin{aligned} \text{size} &= O(3^r/r), \text{depth} \leq (3 \cdot \log_6(3))r + O(r/\log r), \\ \text{size} &= O(3^r/r), \text{depth} \leq (3/2)r + O(r/\log r). \end{aligned}$$

Neither fixed router is claimed capacity-optimal. A radius-one search found no R10 insertion into the frozen R9, which is only a bounded NO_HIT.

6.8 Recursive signed discriminator routers

415 The fixed R9 is not the final routing object. On the Boolean subalgebra define positive and negative depth-one cells

$$\begin{aligned} P_1(x; c0, c1) &= d(x, c0, c1), \\ N_1(x; c0, c1) &= d(c0, x, c1). \end{aligned}$$

The exact programs are

cell	projection mode	constant 0	constant 1
P_1	(0, 0) returns x	(1, 0)	(0, 1)
N_1	(0, 1) returns $1 - x$	(0, 0)	(1, 1)

Recursively set

$$\begin{aligned} P_h &= d(P_{h-1}, N_{h-1}, P_{h-1}), \\ N_h &= d(N_{h-1}, P_{h-1}, N_{h-1}). \end{aligned}$$

For a target in the left, middle, or right branch group, program the three children respectively as

420 (projection, constant 0, constant 0),
 (constant 0, projection, constant 1),
 (constant 0, constant 0, projection).

At a positive parent the resulting expressions are

$$\begin{aligned} d(x, 0, 0) &= x, \\ d(0, 1 - x, 1) &= x, \\ d(0, 0, x) &= x. \end{aligned}$$

425 The same transitions at a negative parent return $1 - x$. Constants lift by $d(c, c, c) = c$.

Theorem 6.7 (exact strong signed-router family). For every $h \geq 1$, there are d-only full trees P_h, N_h with

capacity	$q_h = 3^{h-1}$,
branch leaves	q_h ,
address-control leaves	$2q_h$,
discriminator nodes	$(3^h - 1)/2$,
dependency depth	h .

For every target, P_h returns the target branch and N_h its Boolean complement; both signs also have constant-zero and constant-one modes. Every branch path in P_h has even middle-edge parity, while every branch path in N_h has odd parity.

Proof. The semantic modes follow from the preceding induction. The recurrences

$$\begin{aligned} q_h &= 3q_{h-1}, \\ C_h &= 3C_{h-1}, \\ S_h &= 1 + 3S_{h-1}, \\ D_h &= 1 + D_{h-1} \end{aligned}$$

start at $q_1 = 1, C_1 = 2, S_1 = D_1 = 1$ and give the displayed counts. The middle child swaps P and N, while the outer children preserve the sign, proving the path-parity statement. No NOT or signed-leaf constructor occurs in the tree. $\square$

Thus depth four already gives an exact R27 with 40 d nodes, 27 branches, and 54 controls; depth six gives R243. At fixed h the routing coefficient is $h/(h-1)$, tending to one. The family theorem has been independently derived and replayed and has also been formalized in Lean in the exact d-only grammar. The grammar's lack of a constant or NOT constructor is enforced by its inductive constructors, not by the historical `OriginalSignature` predicate, which is true of every inhabitant. A later additive Lean layer re-proves the address/control cardinalities and $q_h = 3^{h-1}$ with kernel-reduced decide and induction, avoiding the predecessor file's native-evaluator checkpoints.

The representation bridges must still be charged. On a nonbinary slice, $A = 2$ supplies legal names $u(A)$ and $u(u(A))$; two sibling copies of the router carry the Boolean planes and are decoded once. On the binary cube a logical relative program bit p is address-only, but its physical representative is x_0 when $p = 0$ and $u(x_0)$ when $p = 1$. Those physical wires are payload-relative. Their legality comes from simultaneous-complement equivariance, not from treating them as absolute constants or literally payload-independent controls.

Choosing $k = \lfloor \sqrt{r} \rfloor$, $h = k + 1$, and $q = 3^k$, and routing every short residual chunk first, gives the following audited compiler implication.

Conditional Corollary 6.8 (growing-router compiler). Assume the coordinate-selector, global-anchor, two-plane representation, binary-orbit, decoder, and outer-glue lemmas used by the preceding compiler. Then every $f \in \text{CT}_r(Q)$ has one original-signature shared DAG with

$$\begin{aligned} \text{size} &= O(3^r/r), \\ \text{depth} &= r + O(\sqrt{r}). \end{aligned}$$

460 The residual-first qualification is necessary for this ledger. A residual-last schedule has an explicit coefficient-breaking counterexample at $r = 512$. An independent accounting audit checks the repaired integer schedule for every $64 \leq r \leq 10000$, including all controls, both planes, padding, the binary branch, anchor, decoder, and glue. This is an audited conditional composition, not a Lean proof of the end-to-end compiler.

465 6.9 Sibling-shared program vectors and logarithmic overhead

The signed family exposes a second bottleneck: compiling $2q$ program slots one at a time costs $\Theta(q \log q)$ for a capacity- q router. The slots instead form a classical first-nonpropagating prefix state. The balanced-prefix idea is standard [29,30]; the local content here is its charged constant-free $\{d, u\}$ realization and exact sharing census.

470 A **multi-output shared DAG** is an acyclic original-signature operation graph with a finite ordered list of distinguished roots. Its size is the number of operation nodes in the union of their ancestor subgraphs, counted once; inputs are free and fan-out is unrestricted. Substitution adjoins that union once and permits arbitrary references to its roots. The final compiler still has one output root and uses the single-output convention of Section 2.

475 Write a physical branch and target as $p, t \in \{0, 1, 2\}^w$, and put $q = 3^w$. Let $B_p(t)$ be one when the first mismatch exists and is not $(t_j, p_j) = (1, 2)$, and let $G_p(t)$ be one when the first mismatch is exactly that exceptional pair. Equality is the residual state $(B, G) = (0, 0)$. The rails are disjoint. For consecutive word blocks U, V , first-mismatch ownership gives the associative laws

$$\begin{aligned} B_{UV} &= d(B_U, G_U, B_V), \\ G_{UV} &= d(G_U, B_U, G_V). \end{aligned}$$

Writing $N = \neg B$, the complement rail co-produces in the same layer:

$$N_{UV} = d(G_U, B_U, N_V).$$

480 On the nonbinary branch put $one = u(A)$ and $zero = u(one)$. The complete one-digit base uses four non-name operation nodes:

$$\begin{aligned} \text{physical0} : B &= d(x, A, one), G = zero, \\ \text{physical1} : B &= d(one, x, zero), G = zero, \\ \text{physical2} : B &= u(d(x, A, one)), G = d(x, A, zero). \end{aligned}$$

The final control pair is (B, G) in a positive cell and (G, N) in a negative cell. Cell sign is the root sign xor the parity of the middle digits of p . Constant mode c uses $(1 - c, c)$ in a positive cell and (c, c) in a negative cell. No payload wire enters this address program.

485 Two identities create the decisive shared-DAG reduction:

$$\begin{aligned} G_{as} &= G_a \text{ when the physical suffix contains no digit 2,} \\ N_{a1} &= G_{a2}. \end{aligned}$$

The first reuses the left gain root whenever the right gain rail is zero. The second points every eligible not-B rail to an already materialized sibling gain root. Both are ordinary root reuse under free fan-out, not free gates.

Put $a = \lceil w/2 \rceil$, $b = \lfloor w/2 \rfloor$, and $A_0 = 3^a$. Let $R(w)$ count the full (B, G) library and $E(w)$ the full
 490 (B, G, N) suffix library, excluding the two names. Exact structural counting gives

$$\begin{aligned} R(1) &= 4, \\ R(w) &= R(a) + R(b) + 3^w + A_0 \cdot (3^b - 1)/2, \\ E(1) &= 4, \\ E(w) &= R(a) + E(b) + 2 \cdot 3^w - 3^{w-1} + A_0 \cdot (3^b - 1)/2. \end{aligned}$$

At the top merge, the mixed/gain overlap has size

$$\begin{aligned} O_P(w) &= (3^{w-1} + 1)/2, \\ O_N(w) &= (3^{w-1} - 1)/2. \end{aligned}$$

Thus, for $w \geq 2$, the exact sizes of the displayed P and N constructions are

$$\begin{aligned} S_P(w) &= 2 + R(a) + E(b) + A_0 \cdot (3^b - 1)/2 + 3^w - O_P(w), \\ S_N(w) &= 2 + R(a) + E(b) + A_0 \cdot (3^b - 1)/2 + 3^w - O_N(w). \end{aligned}$$

The direct bases are $S_P(1) = 5$ and $S_N(1) = 6$.

Theorem 6.9 (sibling-shared program vector). Let $A = 2$. For either fixed P or N root sign and
 495 every $w \geq 1$, all $2q$ projection controls of the capacity- $q = 3^w$ signed router have one legal address-only scalar $\{d, u\}$ multi-output shared DAG satisfying

$$\begin{aligned} 3 \cdot S(w) &\leq 4q + 15 \cdot 3^{\lceil w/2 \rceil}, \\ S(w) &\leq 7q/3, \\ D(w) &\leq 3 + \lceil \log_2 w \rceil, \end{aligned}$$

including the two shared name nodes and measured above raw anchor/address inputs. Consequently $S(w) = (4/3 + o(1))q$. Width zero uses only the two names.

Proof. Induction on the balanced split gives $R(w) \leq 20 \cdot 3^w/9$ and $E(w) \leq 26 \cdot 3^w/9$; after substituting
 500 these inequalities in the exact top count, the residual half-width term is at most $5 \cdot 3^{\lceil w/2 \rceil}$. This proves the first display. It implies $S \leq 7q/3$ from width four onward, and widths one through three are direct bases. The one-digit rails have depth at most three, and each balanced merge adds one level, proving the depth bound. $\square$

The bound has a matching leading lower term in this declared local model. The gain family has
 505 $(q + 1)/2$ distinct scalar functions, the sign-selected B/N family has q , and the only cross-family identities are $N_{a1} = G_{a2}$. Their counts are $(q/3 + 1)/2$ for P and $(q/3 - 1)/2$ for N. Hence the requested controls comprise exactly $4q/3$ distinct functions for P and $4q/3 + 1$ for N. Every one is Boolean-valued, whereas the free terminals are the constant anchor $A = 2$ and Q-surjective address

coordinates. No requested control is a free terminal, and one scalar operation node can root only one new scalar function. Therefore the leading local size constant is exactly $4/3$.

This lower bound does not apply to an arbitrary integrated router that avoids exposing these exact controls, to simultaneous P and N materialization, to formulas, or to bounded fan-out. It does not determine the finite-width exact minimum.

Two independent no-author-import reconstructions check the semantics, recurrences, sharing identities, and lower census. Lean proves the rail algebra, one-digit bases, zero-gain reuse, sibling identity, final control decoding, exact arithmetic recurrences, the all-width envelope, the uniform upper bound, and the matched sharp depth $3 + \lceil \log_2 w \rceil$. The formal cost object is still a declared recurrence ledger rather than a serialized hash-consed DAG; its refinement link is the executable construction plus independent structural audit. The distinct-output lower census is only partially formalized: the bad/not-bad families are injective, but the gain quotient and final signed inclusion-exclusion remain open in Lean. A direct one-wire three-state encoding with merge $d(S_U, A, S_V)$ is also valid, but its per-cell decoding is strictly more expensive than the sibling-shared two-rail construction.

To apply this vector to the compiler, fix $r \geq 64$, put

$$\begin{aligned} L &= 4 + \lceil \log_3(r^2) \rceil, \\ H &= r - L, \\ M &= 3^{\lceil \log_3 H \rceil}, \\ b &= \log_3 M, \\ s &= r - b, \\ P &= 3^s, \end{aligned}$$

and let $C = \lceil \log_2 r \rceil$. Proposition 3.3 supplies a total coordinate selector $\sigma : Q^r \rightarrow \{0, \dots, r-1\}$ that is invariant under simultaneous complement on the Boolean cube. Lemma 6.2 supplies the ordered balanced anchor A , with $A = 2$ off that cube and $A = x_0$ on it.

Lemma 6.10a (nonbinary two-plane compiler). On $A = 2$, there is one legal original-signature shared DAG whose output is $x_{\sigma(x)}$. Inclusive of the anchor, two generated names, one decoder, and the three nodes reserved for the final glue, its operation-node count is at most

$$N_{\text{NB}} = (3M - 1)3^M + (3P - 1) + S_P(b) + S_P(s) - 2 + 4(r - 1) + 2 + 3,$$

and hence is below $(32 + 1/27 + 1/2)3^r/r$. Its output before the final glue has depth at most $r + 4C + 7$.

Proof. Write each input as $x = (p, l)$ with $p \in Q^s$ and $l \in Q^b$. For a fixed prefix p , define the local table

$$g_p(l) = (p, l)_{\sigma(p, l)}.$$

Materialize the universal library of all 3^M Q-valued tables on Q^b . Encode table values by the two legal Boolean planes

$$0 \rightarrow (0,0), 1 \rightarrow (0,1), 2 \rightarrow (1,0).$$

Each table uses two capacity-M P routers, controlled by l ; every table and both planes share the same width-b vector from Theorem 6.9. For each prefix p , take the two roots belonging to g_p . Two capacity-P P routers, controlled by p and sharing one width-s vector, select those roots. Decode once with

$$\text{Dec}(\text{high}, \text{low}) = d(d(\text{high}, \text{one}, A), \text{zero}, \text{low}).$$

540 Its values on 00, 01, 10 are respectively 0, 1, 2; the unused code 11 is irrelevant. Thus the output is exactly $g_p(l) = x_{\sigma(x)}$. Every control is a term over the raw address and A ; the names $\text{one} = u(A)$ and $\text{zero} = u(\text{one})$ are charged nodes, not nullaries. Each vector is adjoined once and reused by every table, plane, and router that names it.

Two capacity-M routers have $3M - 1$ discriminator nodes, and two capacity-P routers have $3P - 1$. 545 This gives the displayed exact upper ledger. Since $\lceil \log_3(r^2) \rceil \leq 2 \log_3(r) + 1$ and $5 + 2 \log_3(r) < r/2$ from $r = 64$ onward, $H > r/2$; together with $H/3 < M \leq H$, this gives $M > r/6$. The two-logarithm reserve also gives

$$3^M \leq 3^r / (81r^2).$$

Consequently the local routers cost less than $(1/27)3^r/r$, while the prefix routers and width-s vector cost less than

$$(3 + 7/3)P < 32 \cdot 3^r/r.$$

550 The width-b vector and fixed overhead are below $7r$ nodes. The elementary inequality $14r^2 < 3^r$, valid from $r = 64$, puts them below $(1/2)3^r/r$.

The anchor has depth $3C$. With $D_P(w) \leq 3 + \lceil \log_2 w \rceil$, the two plane depths before decoding satisfy

$$\begin{aligned} D_{\text{local}} &\leq 3C + D_P(b) + b + 1, \\ D_{\text{prefix}} &\leq \max(D_{\text{local}}, 3C + D_P(s)) + s + 1. \end{aligned}$$

Adding the two decoder levels and using $b + s = r$ gives $D_{\text{NB}} \leq r + 4C + 7$. $\square$

555 The binary cube cannot use the absolute names just used. Its compiler is instead complement-relative.

Lemma 6.10b (binary complement-relative compiler). On the Boolean cube there is a legal original-signature shared DAG whose output is $x_{\sigma(x)}$. For $r \geq 64$ its increment over the already present anchor is below $(1/2)3^r/r$, and its intrinsic output depth is at most $r + 4C + 7$.

560 **Proof.** Put $z_i = 0$ when $x_i = x_0$ and $z_i = 1$ otherwise. The word z and the selector index are invariant under simultaneous complement. Choose

$$k = \lfloor \text{sqrtr} \rfloor, q = 3^k, w = \lfloor \log_2 q \rfloor.$$

Partition the $r - 1$ relative coordinates into chunks of width at most w , placing the short residual chunk first. At a chunk of width c , fix an injection $\iota_c : \{0, 1\}^c \rightarrow \{0, 1, 2\}^k$ into the first 2^c branches of a capacity- q P router, and point every unused branch to the last live child. Each logical control table $f : \{0, 1\}^c \rightarrow \{0, 1\}$ is the corresponding Theorem 6.9 P-control coordinate for $\iota_c(z)$. It is installed as
 565 the physical, complement-relative wire $x_0 \text{ xor } f(z)$ and realized from leaves $x_0, u(x_0)$ by the decision node

$$\text{Sel}(x_i, x_0, E, D) = d(d(x_i, x_0, E), d(x_i, x_0, D), D),$$

which returns E when $x_i = x_0$ and D otherwise. One physical control therefore costs at most $3(2^c - 1)$ nodes and depth $2c + 1$. All router instances at a level share the same $2q$ controls. On the representative orientation $x_0 = 0$, these are the ordinary program bits. At the terminal prefix z , use
 570 leaf $x_{\sigma(0,z)}$; the routed leaf is therefore the requested coordinate. Complementing the input complements every payload and physical control; self-duality of d on $\{0, 1\}$ and complement invariance of σ then prove the other orientation. No absolute Boolean constant has been used.

If I is the total number of router instances and the chunk widths are c_j , residual-first ordering gives $I < 2^{r-1}$. Hence

$$\begin{aligned} B_{\text{router}} &= ((3q - 1)/2)I < (1/4)3^r/r, \\ B_{\text{control}} &= \sum_j 6q(2^{c_j} - 1) < (1/4)3^r/r. \end{aligned}$$

575 For completeness, the first inequality follows from $3rq2^r < 3^r$; the second follows from $B_{\text{control}} < 6rq^2$ and $24r^2q^2 < 3^r$. Since $q \leq 3^{\sqrt{r}}$, these reduce to positivity of

$$\begin{aligned} F_1(r) &= (1 - \log_3 2)r - \sqrt{r} - \log_3(3r), \\ F_2(r) &= r - 2\sqrt{r} - \log_3(24r^2). \end{aligned}$$

Both are positive at 64. For $r \geq 64$,

$$\begin{aligned} F'_1(r) &= 1 - \log_3 2 - 1/(2\sqrt{r}) - 1/(r \ln 3) > 0, \\ F'_2(r) &= 1 - 1/\sqrt{r} - 2/(r \ln 3) > 0, \end{aligned}$$

so the bounds hold for every real $r \geq 64$, hence every integer arity in scope. The sole $u(x_0)$ leaf is an existing anchor subnode and is not charged twice.

580 If m is the number of chunks, the intrinsic depth is at most

$$D_B \leq (k + 1)m + 2w + 1.$$

For $k \geq 8$, $3k/2 \leq w < 8k/5$. Indeed, $2^{19} < 3^{12}$ and $3^5 < 2^8$ give $19/12 < \log_2(3) < 8/5$, and the lower fractional margin exceeds one half already at $k = 8$. Since $m \leq 2(r - 1)/(3k) + 1$ and $r < (k + 1)^2$, this yields

$$\begin{aligned}
D_B &< 2r/3 + (73/15)k + 8/3 \\
&\leq 2r/3 + (73/15)\sqrt{r} + 8/3 \\
&\leq r + 4C + 7.
\end{aligned}$$

For the last inequality, use $C \geq \log_2 r$. The difference is at least

585

$$r/3 + 4\log_2(r) + 13/3 - (73/15)\sqrt{r}.$$

It is positive at 64, and its derivative is at least $1/3 - 73/(30\sqrt{r}) > 0$ there and thereafter. The displayed upper ledger is $\log_3(2)r + O(\sqrt{r})$, so this branch is asymptotically below r . $\square$

590

Put $zero = u(u(A))$, let B and N denote the binary and nonbinary outputs, and define

$$\text{Glue}(A, B, N) = d(d(zero, A, B), d(zero, A, N), N).$$

The glue adds exactly three discriminator nodes and two levels. On the Boolean cube $zero = A$, so it returns B ; off the cube $A = 2$, $zero = 0$, so it returns N .

Theorem 6.10 (integrated parallel-program compiler). Every $f \in \text{CT}_r(Q)$ has one parameter-free original-signature term DAG with

$$\begin{aligned}
\text{size} &= O(3^r/r), \\
\text{depth} &= r + O(\log r).
\end{aligned}$$

595

For every $r \geq 64$, one such DAG has

$$\begin{aligned}
\text{size} &< 34 \cdot 3^r/r, \\
\text{depth} &\leq r + 4 \cdot \lceil \log_2 r \rceil + 9.
\end{aligned}$$

Proof. Apply Proposition 3.3 to obtain σ , construct the common anchor, take the union of the two DAGs in Lemmas 6.10a and 6.10b, and adjoin the glue. The nonbinary ledger already reserves the unique glue. Therefore

$$\text{size} / (3^r/r) < 32 + 1/27 + 1/2 + 1/4 + 1/4 < 34.$$

600

Free fan-out makes this a union of ancestors, not a formula expansion: each program vector, table root, and control root is installed once. The depth is the maximum, not the sum, of the anchor/-name path, the binary path, and the nonbinary path, followed by two glue levels. The two lemmas give the displayed bound. Theorem 6.1 handles the finite range $r < 64$. $\square$

605

Together with Theorem 4.2, Theorem 6.10 matches the leading depth coefficient one. It does not determine the optimal additive term, ordinary-tree complexity, a bounded-fanout bound, or an exact global size constant. A standalone no-author-import implementation independently reconstructs the anchor, signed router, sibling-shared vector, both compiler branches, decoder, same-DAG union, and glue; it checks effective mutations and exact integer ledgers through arity 16,384. The integrated construction has not yet been serialized and proved in Lean, and this internal audit is not external peer review.

7. Compiler progression and bounded validation

7.1 One common arity-nine target

The compilers were compared on the same fixed arity-nine reduction selector, evaluating all $3^9 = 19,683$ input rows. The reported node counts include variables, and reported depths put variables at depth one. They describe the emitted DAGs under that executable convention; they are not $C_r(f)$, $D_r(f)$, exact minima, or lower bounds.

compiler schedule	distinct reachable DAG nodes	reported depth
no-wasted-leaf first-2 classifier	58,331	37
Gray local coding	19,329	179
layered prefix library	18,327	45
recursive subcube library	18,135	37
global anchor and depth-three router	23,807	41

The table illustrates different goals. Gray local coding produces the first large size reduction but serializes the library. Layering removes that chain. Recursive subcubes give the best bounded result in this test. The global-anchor compiler is asymptotically shallower but is worse at arity nine.

The structural analytic bounds are equal at arity 16; the global bound is smaller than $4r$ from arity 20 onward. At arities 32, 64, 128, 256, the recorded depth-bound savings are 13, 42, 103, 228, respectively. These are arithmetic consequences of the proved formulas, not performance claims about a materialized large circuit.

The later routing stages are theorem-level schedule comparisons, not additions to the materialized arity-nine benchmark above:

routing stage	same-DAG size	depth ceiling	evidence status
historical R6	$O(3^r/r)$	$(3 \cdot \log_6(3))r + O(r/\log r)$	manuscript proof plus bounded replay and audit
historical Boolean-plane R9	$O(3^r/r)$	$(3/2)r + O(r/\log r)$	manuscript proof plus bounded replay and audit
recursive signed family	$O(3^r/r)$	$r + O(\sqrt{r})$	exact family independently audited and Lean-checked; historical repaired compiler stage

routing stage	same-DAG size	depth ceiling	evidence status
sibling-shared vector	$O(3^r/r)$	$r + O(\log r)$	local $(4/3 + o(1))q$ vector plus matching leading scalar-output lower bound; integrated compiler independently reconstructed

R9 is therefore a preserved historical stage, not the current ceiling.

7.2 Finite checks

The deterministic checkers currently establish the following bounded facts.

- The exact category census agrees with the formula through arity eight.
- 630 • All 1 and 32 semantic conservative operations at arities one and two are directly enumerated and compiled.
- The arity-three formula evaluates to 23,887,872.
- All 128 compatible arity-two selector-index tables are compiled and checked; the larger number reflects nonunique selector representations, not additional semantic operations.
- 635 • Fixed-seed full-table evaluation is performed through arity seven.
- Every row of the arity-nine reduction selector is evaluated.
- Gray paths through block length four, layered selector censuses through $M = 8$, representative recursive-subcube censuses, and asymptotic plan arithmetic through arity 64 are checked.
- The absorber is checked on all nine input pairs; the router is checked on all 81 valuations; balanced anchors are checked through arity seven; and global structural depth arithmetic is checked through arity 256.
- 640 • The programmable routers are checked on all 4,374 six-way cases, 896 seven-way cases, and 896 complement-relative cases. The composed compiler covers all 128 compatible arity-two selector tables plus fixed-seed tables through arity five, and its library and structural recurrences are checked through 128 points and arity 4,096, respectively.
- 645 • The Boolean R9 is checked on all 4,608 projections and 9,216 relative-orientation cases; the two-plane transfer is checked on all 177,147 target/payload cases. Its composed compiler covers 135 tables and 1,641 complete tuples through arity five, with the library recurrence checked through 256 points and structural depth arithmetic through arity 4,096.
- 650 • The recursive signed family is proved by induction, independently checked through direct truth tables and discriminator ROBDDs, mutation-tested on every R27 target, and Lean-checked for every depth, sign, mode, target, and Boolean branch valuation. The independent compiler audit checks every integer arity 64..10000 after the residual-first repair.
- 655 • The historical E/G vector is checked on 1,200,114 target/physical/sign pairs, 18 dependency sets, 11,356 concrete router cases, 262 direct-discriminator ROBDD modes, and 232 padding cases. Its author ledger checks every integer arity 64..4096; an independent audit extends the arithmetic replay through arity 16,384 and materializes both signs through width nine.

- The sibling-shared vector has two independent no-author-import reconstructions. The broader algebraic replay checks 5,978,710 control-pair identities across five constructions through width six, exact structural counts through width ten, recurrence arithmetic, ROBDD and concrete router semantics, and effective mutations. Its normal and optimized receipts are byte-identical. The separate referee reconstruction derives the exact recurrences and scalar-output census without importing the author builder.
- Lean proves the optimized rail algebra, disjointness, one-digit bases, one-layer block composition, zero-gain reuse, sibling identity, final controls, exact sibling recurrences, the all-width $3S \leq 4q + 15 \cdot 3^{\lceil w/2 \rceil}$ envelope, and uniform $S \leq 7q/3$. The formal cost is a recurrence-level ledger rather than a serialized hash-consed DAG; the independent structural implementations supply that refinement evidence. Lean now proves the sharp local depth. It does not yet prove the complete distinct-output lower census or the end-to-end $r + O(\log r)$ compiler.
- An additive manuscript-repair Lean layer re-proves router cardinalities with kernel-reduced decide, replaces vacuous signature predicates by exhaustive no-nullary-constructor catalogs, and proves the anchor cell, outer glue, and valid two-plane decoder identities. It still does not formalize the full ordered anchor DAG or integrated compiler.
- The independent compiler-bridge reconstruction imports no author implementation. It checks 3,279 anchor rows, 597,870 target/physical vector identities, 4,632 concrete router cases, all 128 compatible arity-two selectors plus four deterministic arity-three selectors through both branches and the glue, and every exact integer ledger from arity 64 through 16,384. Normal and optimized outputs are byte-identical. Its written induction and inequalities, not that bounded sweep alone, support Theorem 6.10.
- Mutation controls detect omitted complement quotienting, spurious all-2 choices, complement asymmetry, corrupted Gray paths, invalid block caps or schedules, incorrect selector censuses, dropped depth corrections, wrong absorbers, constant swaps, incomplete tables, and binary-anchor scope errors.
- Programmable-routing controls additionally reject effective gadget mutations, illegal absolute binary controls, omitted or per-node control accounting, sparse full-code accounting, omission of the binary depth branch, and a non-complement-invariant selector table.
- Boolean-plane controls additionally reject serializing the planes, repeating the decoder at every level, omitting the decoder charge, and moving from per-level to per-node controls.

These computations validate implementations, bounded identities, and arithmetic. They do not prove the all- r theorems or publication novelty.

7.3 Replay receipts

Normal and optimized Python runs agree at the following semantic hashes.

receipt	semantic SHA-256
runs/quasiprimal_conservative_term_count/summary.json	f7f1758972b68abd8aba60ae547cb899c5d9376cced7cba449c34a2697e001b8
runs/quasiprimal_local_coding_compiler/summary.json	4aaf27d678847aa0455ae8be552a165c3e9ec08c007031878f6aa40b0794ed89
runs/quasiprimal_layered_local_coding_compiler/summary.json	b42fa56c1e78e670e2a436181337957309a0b2a658b6f539486b85cedc482d54

receipt	semantic SHA-256
runs/quasiprimal_subcube_local_coding_compiler/summary.json	f9d89ec9829c3abef05d1cdc17ef811330674bd056d90c3579a8f32b0b2eb7b8
runs/quasiprimal_global_anchor_routing/summary.json	fb5ca7bf56d9aba9fdb0f69d1f1355c92d56d4f9306ee08a922bfc770ef60501
runs/quasiprimal_programmable_projection_routing/summary.json	e663534756cf67ffb12fd9ae3fe722c465bd70cd91b73af1fb8084af1c0706dc
runs/quasiprimal_boolean_r9_routing/summary.json	e4a09ef72986c1221c28e80d8227879a9e0aa0f93679e49e9f750f7e65977fce
research/tournaments/2026-08-13-semantic-router-frontier/lanes/fused/summary.json	4852080965ac89f54f436401950ca22ecc0dd7887593b5c01dcadcaeb883ea2a
research/tournaments/2026-08-13-semantic-router-frontier/audits/strong_family/receipt.json	ac566d0b716575ff693bc3a1f125283c764ef608a04c183a17293fe66adc4cb2
research/tournaments/2026-08-13-semantic-router-frontier/audits/variable_compiler/receipt.json	d81d0bd6b979ec0df4270fd60f04468884de6e286aea450ba28eed9897e51870
research/tournaments/2026-08-13-semantic-router-frontier/lanes/program_vector/receipt.json	9699a3fe058369ac22a4a4d8050d07825cc80112cbc4c601a85914100db13f88
research/tournaments/2026-08-13-semantic-router-frontier/audits/program_vector/receipt.json	4922df2221b3e6195d7b1af3a3ac770ae92a8aed10843430b9cffb20959c58ea
research/tournaments/2026-08-13-program-vector-optimization/lanes/native_scan/receipts/receipt.json	3b30a1ad6de10d39705d65edc3b27b8fd22acf361d2f7bd68f47d4d459e6d2ac
research/tournaments/2026-08-13-program-vector-optimization/lanes/referee/kuhn_boolean_rail_audit/receipt.json	1be842ffaca9c0510834e0eac1ee38f0acc6c310d15bb954c1a76c24dcc6f342
research/tournaments/2026-08-13-program-vector-optimization/lanes/algebraic/receipt.json	647f91c6c85173bbcb275e43a2301a67c61a7fd4d703b4720664a1341d7277aa
research/tournaments/2026-08-13-program-vector-optimization/audits/compiler_bridge/receipt.json	73a43b1e4292b56824e82a471833e69252c52f212463f182a90df63703eec23e

The formal checkpoints are source- and receipt-hash bound:

formal source	source SHA-256	receipt SHA-256
lanes/formal/StrongSignedRouter.lean	687a77ed1f3b0bfe2a540bc670f6db942e15bddc339ccfbc effba9699766227a	e984f5cf50bc23489fe7bd832f08c075f61abf66885c6b0736735420eb934662

formal source	source SHA-256	receipt SHA-256
lanes/formal_program_vector/ProgramVector.lean	c2f477edd110c4df96f3c30f31f02de09af93045720babfd38dd2dd79a5573dd	5e79e4eb49af361fbed1c8962a67583a48e532526698da584b8825d55136cffa
lanes/formal_program_vector_cost/ProgramVectorCost.lean	ea516b085cdedd3f0ee70f83a9d0240df55e7e68cf0ad8ce77558efd91db55d2	4919835f32d8afdfc48f72979edf4ad4ef0b1f8b16271ed9858a60750d2b6211
program-vector-optimization/lanes/formal_optimized/DirectRail.lean	57963dadfb40cef30d94f630126f3179c55e98bea68e5974daa8e91913c404bd	cd5f1a49799726c3009748b5634a30cc154dd05d28440881df07404738c4e42e
program-vector-optimization/lanes/formal_optimized/SiblingShared.lean	8eb02a3e2e5a4f7531d740b704422d09724376821f83b921aec98b77a0277e77	cd5f1a49799726c3009748b5634a30cc154dd05d28440881df07404738c4e42e
program-vector-optimization/lanes/formal_gap_closure/DepthAndCensus.lean	ea75a228cc1316f0c53cff397409d4b9f6be4202611e29be28488a81f9a7be8c	7dcae0bbc3b3ced6d244886be01f3090a382859a6baac44906a31bb87b526a79
program-vector-optimization/lanes/formal_manuscript_repairs/ManuscriptRepairs.lean	4b082559f9bdcea0dc8518992c9c9a87960f2a33a289a0d3cf7ad55347b3ced7	cb9e6f0d0bbbee03738e39a541e305a171e20a65fbac82b70e8f4faac59e16631

8. Relation to prior work

695 This draft intentionally separates classical ingredients, directly adjacent work, and the still-unknown originality of the exact conjunction.

8.1 Universal algebra

700 Pixley’s discriminator and quasi-primal results [1,2] supply the internal-isomorphism characterization used in Proposition 3.1. Baker–Pixley interpolation [3] and Quackenbusch’s extension hierarchy [4] form the wider structural background. None of those results is claimed here. Modern discriminator-clone and conservative-clone work, including [14], must also be checked in full before submission.

8.2 Shannon synthesis and multivalued logic

705 Shannon’s circuit-counting method is classical [5]. Lupanov introduced the local-coding principle used in the upper bound [6]. Yablonskii’s functional constructions in k -valued logic establish a much broader historical setting [7]. Orlov studies realization and basis-sensitive circuit and formula Shannon functions for k -valued bases [8,27]. Kochergin establishes linear Shannon-depth behavior for k -valued functions over arbitrary complete finite bases [9]. Safin studies depth versus complexity in precomplete multivalued classes [10]. Korshunov’s survey maps local coding, 710 Shannon effects, leading coefficients, formulas, and depth [13]. Gashkov’s primary 1978 theorem gives Boolean formulas over $\{AND, OR, NOT\}$ with depth $n - \log_2(\log_2 n) + O(1)$ while retaining formula-Shannon size $\Theta(2^n / \log n)$ [28].

The present parameter-free basis is incomplete because every term preserves $B = \{0, 1\}$. Therefore the scopes are not textually identical. That difference is not novelty evidence: the present theorems, including the signed recurrence and sibling-shared vector, may still be expected specializations or consequences of older closed-class complexity, local-code decoding, switching-term, multiplexer, or universal-circuit machinery. Dual-rail or two-rail representations are also classical in logic synthesis; Ishiura gives a two-rail-input/two-rail-output BDD synthesis construction in a different basis and objective [19], and Backes uses dual-rail encodings of ternary semantic values for verification-oriented circuit transformations [20]. These do not establish or refute the fixed-Q theorem, but they prevent claiming the representation change itself as novel. Full-text Russian and international review is required. Tarasov’s work on depth versus complexity for many-valued functional systems is another nearby line that must be compared directly before submission [17,18].

The first-mismatch state law in Theorem 6.9 is also a classical kill/propagate/generate-style prefix monoid. Ladner–Fischer parallel-prefix computation [29] and Brent–Kung generate/propagate networks [30] are therefore explicit methodological ancestry. The paper does not claim the monoid or its balanced evaluation as new. The potentially local contribution is the exact constant-free discriminator embedding, the sibling root identities, and the matching $4q/3$ scalar-output census under this paper’s cost model.

8.3 Simultaneous size and depth, and multiplexer depth

Lozhkin gives Boolean results on formulas simultaneously approaching Shannon size and depth [11] and, more recently, exact basis-sensitive depth results for multiplexers with few selector lines [12]. Kochergin’s finite-complete-basis theorem [9] already makes linear Shannon depth a classical phenomenon; the point here is one incomplete parameter-free basis, not the order $\Theta(r)$ by itself. McColl and Paterson give a classical leading-one depth shape for all Boolean functions in a richer basis [24]. Gashkov’s source proof uses a Lupanov parallel-series representation, but a checked transfer ledger shows that dense Booleanization gives leading depth $\log_2(3)r$ and source-witness size $\Theta(3^r / \log r)$, so it does not directly supply the present same-DAG order $O(3^r/r)$ or coefficient one [28]. This nontransfer is not novelty evidence: a native Q-valued Lupanov–Gashkov construction remains open, and Lupanov’s separate delayed-circuit model still requires a primary-source model comparison. Valiant’s universal circuits and Cook–Hoover depth-universal circuits establish programmable control and joint size/depth routing as classical themes [21,22]. Holmgren and Rothblum give modern linear-size shared multiselection circuits in a different Boolean model [23]. Those models do not immediately translate to this incomplete ternary term clone, but they make broad novelty claims about programmable controls, shared selection, simultaneous synthesis, or leading-one depth unsafe. The unresolved comparison is the exact conjunction of the fixed no-nullary signature, signed P/N counts, charged sibling-shared materialization, and same-DAG compiler. Because d has arity three on a three-element address alphabet, a leading depth coefficient one is the natural information rate; any eventual novelty claim must rest on the exact incomplete-basis realization, not surprise at that coefficient.

8.4 Chinese-language adjacency

The public-index search located Wu and Chen’s 1985 ternary sequential-circuit work with three-rail output [15] and Xu, Guan, and Zhang’s 2013 ternary reversible-logic synthesis algorithm [16]. A focused follow-up also located Zhang, Chen, and Wang’s BDD-to-MUX area/delay optimization [25] and He, Gong, and Wei’s shared multi-output BDD construction [26]. Their algebras, reversibility conditions, and physical, empirical, or gate-cost objectives differ from original-signature

Q -term DAGs, but the latter two are direct adjacency for shared selector structures.

No direct match was verified in the bounded search for the exact fixed- Q census, the d-only signed P_h/N_h family, the sibling-shared $4q/3$ local vector, or the integrated same-DAG $O(3^r/r)$ and $r + O(\log r)$ conjunction. This is only weak negative evidence. Native CNKI/Wanfang full-text coverage, translation variance for clone-theoretic terminology, Russian citation-chain review, paywalled many-valued sources, and expert comparison remain missing.

8.5 Paper-safe positioning

The strongest currently supportable positioning is:

Classical interpolation, counting, local-coding, parallel-prefix, and programmable-selection tools are specialized to one fixed parameter-free conservative term fragment. The exact census and signed-router family are joined by a scalar program-vector interface with matching $4q/3$ leading upper and lower terms in its declared local model; an explicit two-plane and complement-relative construction then gives one same-DAG compiler at counting-scale size and leading-one depth.

Whether that exact specialization or conjunction is publication-novel remains **UNKNOWN**.

9. Provenance, Tau boundary, and explicit nonclaims

The mathematics in this draft concerns one public classical finite algebra and public circuit-complexity methods. The recorded implementations were independently written for OrbitSynthesis and do not rely on Tau source, Tau specifications, private communications, private materials, or an unsigned developer license. This is a provenance statement only. It is not a novelty finding, a patent analysis, a license interpretation, or a noninfringement opinion.

This draft does **not** claim:

- an optimal leading constant for the global Shannon-size function or the integrated all-arity compiler; Theorem 6.9's $4/3$ statement is only the leading constant for one exact scalar-output program-vector interface;
- an exact or optimal additive term beyond the leading-depth coefficient one;
- that the finite Z3 result applies outside its selector-only depth-two grammar;
- an $\Omega(3^r)$ size lower bound;
- an order-optimal ordinary unshared-term-tree bound;
- novelty of Pixley interpolation, Shannon counting, Lupanov local coding, Gray codes, recursive Shannon expansion, layered scheduling, or generic linear depth;
- the same count or bounds for every quasi-primal algebra or every incomplete clone;
- a general $r + \lceil \log_3 r \rceil + O(1)$ lower barrier for every compiler; the dependency argument currently applies only to a restricted exposed-control router cascade;
- Lean extraction of a serialized materialized DAG or formal verification of the integrated compiler, a complete external independent proof review, or peer review;
- publication novelty over Russian, Chinese, or international closed-class complexity literature;
- use of, conclusions about, or limitations of Tau private work;
- rights under the unsigned Tau developer license; or
- patent freedom to operate.

Any commercial implementation, patent filing, categorical novelty statement, or claim-adjacent Tau integration requires separate legal/license review and a current prior-art search.

10. Remaining and quarantined work

The mathematical next steps are:

1. obtain external mathematical review of the independently reconstructed integrated compiler and compare its exact model with the closest many-valued and delayed-circuit literature;
2. extract or serialize the sibling-shared program-vector DAG and formally connect its width-indexed terms, subterm union, recurrence, and compiler substitution in Lean;
3. determine whether Lupanov’s delayed-circuit model, Orlov–Lupanov, Kochergin, multiplexer, universal-circuit, or closed-class results already subsume the exact fixed-signature conjunction;
4. determine the optimal additive term between the counting lower bound and the $r + O(\log r)$ upper bound;
5. determine ordinary-tree complexity and the global, rather than local program-vector, basis-sensitive leading-size constants; and
6. classify fixed finite clones for which parameter-free local coding gives the same order closure.

Boolean-plane R9 and the $7q$ E/G vector remain fully visible historical stages. The exact signed family and sibling-shared $(4/3 + o(1))q$ vector are independently audited; the latter has a matching leading scalar-output lower bound in its declared local model. The complete $r + O(\log r)$ composition has an independent no-author-import reconstruction and is promoted as a manuscript theorem, but not as an end-to-end Lean theorem or an externally refereed result. Novelty, formal integration, practical value, and FTO remain quarantined.

Negative knowledge also remains active: naive ordinary-monotone two-extreme reasoning is false for d’s antitone middle input; residual-last growing-router chunking breaks the size ledger; sequential $\Theta(qw)$ program generation loses the target order; and reserving only one logarithm can make the local library exceed the claimed bound. Solver timeouts, bounded NO_HIT results, and grammar-specific UNSAT results are not upper bounds, lower bounds, or optimality evidence.

11. Reproducibility map

The mathematical source notes for this version are:

- notes/QUASIPRIMAL_CONSERVATIVE_TERM_ENUMERATION.md;
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_SHANNON_COMPLEXITY.md;
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_LINEAR_DEPTH.md;
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_SUBCUBE_DEPTH.md;
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_GLOBAL_ANCHOR_DEPTH.md;
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_PROGRAMMABLE_DEPTH.md;
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_BOOLEAN_R9_DEPTH.md;
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_PARALLEL_PROGRAM_DEPTH.md;
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_OPTIMAL_PROGRAM_VECTOR.md; and
- notes/QUASIPRIMAL_CONSERVATIVE_TERM_INTEGRATED_COMPILER.md.

The frozen theorem, audit, and prior-art packets for the new stages are:

- research/tournaments/2026-08-13-semantic-router-frontier/STATE.md;

- research/tournaments/2026-08-13-semantic-router-frontier/lanes/fused/REPORT.md;
- 840 • research/tournaments/2026-08-13-semantic-router-frontier/audits/strong_family/REPORT.md;
- research/tournaments/2026-08-13-semantic-router-frontier/audits/variable_compiler/REPORT.md;
- 845 • research/tournaments/2026-08-13-semantic-router-frontier/audits/variable_compiler/PRIOR_ART.md;
- research/tournaments/2026-08-13-semantic-router-frontier/lanes/program_vector/REPORT.md;
- research/tournaments/2026-08-13-semantic-router-frontier/audits/program_vector/REPORT.md;
- 850 • research/tournaments/2026-08-13-semantic-router-frontier/lanes/classical_depth_transfer/REPORT.md;
- research/tournaments/2026-08-13-semantic-router-frontier/lanes/formal/REPORT.md;
- research/tournaments/2026-08-13-semantic-router-frontier/lanes/formal_program_vector/REPORT.md;
- 855 • research/tournaments/2026-08-13-semantic-router-frontier/lanes/formal_program_vector_cost/REPORT.md;
- research/tournaments/2026-08-13-program-vector-optimization/lanes/native_scan/REPORT.md;
- 860 • research/tournaments/2026-08-13-program-vector-optimization/lanes/referee/kuhn_boolean_rail_audit/REPORT.md;
- research/tournaments/2026-08-13-program-vector-optimization/lanes/algebraic/REPORT.md;
- research/tournaments/2026-08-13-program-vector-optimization/lanes/formal_optimized/REPORT.md;
- 865 • research/tournaments/2026-08-13-program-vector-optimization/lanes/formal_gap_closure/REPORT.md;
- research/tournaments/2026-08-13-program-vector-optimization/lanes/formal_manuscript_repairs/REPORT.md; and
- 870 • research/tournaments/2026-08-13-program-vector-optimization/audits/compiler_bridge/REPORT.md.

The implementation and deterministic checkers are:

- src/orbitsynthesis/orbit_term_compile_local.py;
- experiments/quasiprimal_conservative_term_count.py;
- 875 • experiments/quasiprimal_local_coding_compiler.py;
- experiments/quasiprimal_layered_local_coding_compiler.py;
- experiments/quasiprimal_subcube_local_coding_compiler.py;
- experiments/quasiprimal_global_anchor_routing.py;
- experiments/quasiprimal_programmable_projection_routing.py;
- 880 • experiments/quasiprimal_boolean_r9_routing.py;
- research/tournaments/2026-08-13-semantic-router-frontier/lanes/fused/check_strong_router_family.py;
- research/tournaments/2026-08-13-semantic-router-frontier/audits/strong_family/audit_strong_family.py;

- `research/tournaments/2026-08-13-semantic-router-frontier/audits/variable_compiler/check_variable_compiler.py`;
- `research/tournaments/2026-08-13-semantic-router-frontier/lanes/program_vector/check_parallel_program_vector.py`;
- `research/tournaments/2026-08-13-semantic-router-frontier/audits/program_vector/audit_program_vector.py`;
- `research/tournaments/2026-08-13-semantic-router-frontier/lanes/classical_depth_transfer/check_transfer_barriers.py`;
- `research/tournaments/2026-08-13-semantic-router-frontier/lanes/formal/StrongSignedRouter.lean`;
- `research/tournaments/2026-08-13-semantic-router-frontier/lanes/formal_program_vector/ProgramVector.lean`;
- `research/tournaments/2026-08-13-semantic-router-frontier/lanes/formal_program_vector_cost/ProgramVectorCost.lean`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/native_scan/check_native_scan.py`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/native_scan/check_optimal_scan.py`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/native_scan/OptimalRailSemantics.lean`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/referee/kuhn_boolean_rail_audit/audit_boolean_rails.py`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/algebraic/check_algebraic_program_vector.py`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/formal_optimized/DirectRail.lean`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/formal_optimized/SiblingShared.lean`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/formal_gap_closure/DepthAndCensus.lean`;
- `research/tournaments/2026-08-13-program-vector-optimization/lanes/formal_manuscript_repairs/ManuscriptRepairs.lean`; and
- `research/tournaments/2026-08-13-program-vector-optimization/audits/compiler_bridge/check_compiler_bridge.py`.

The semantic JSON receipts listed in Section 7 are the bounded-evidence authority for numerical claims. The Lean checkpoint receipts bind the formal sources and toolchain; they do not carry the asymptotic compiler claim. The exact program-vector replay commands and frozen hashes are in `lanes/program_vector/manifest.json`; the independent family commands and hashes are in `audits/strong_family/audit_manifest.json`. The source ledger is `research/SOURCES.md`; the provenance/legal separation is `research/IP_BOUNDARY.md`; and the current status map is `research/RESEARCH_MAP_2026_08_13.md`.

Reproduction must preserve both the normal Python run and the optimized python -O run and require equal semantic hashes. A passing receipt validates only the finite scope named in that receipt.

Declarations

Generative AI and AI-assisted technologies

During exploratory research and preparation of this review draft, the author used OpenAI Codex and other separately invoked large-language-model research agents to propose reformulations and proof strategies, draft and critique mathematical arguments, generate and review Python and Lean artifacts, and assist with organization and copy-editing. Research Kernel and Morph were used as research-orchestration and suggestion systems. Their outputs were treated as untrusted suggestions: promoted claims were required to have human-readable proofs and/or deterministic normal/optimized checkers, independent reimplementations, mutation tests, and scoped Lean kernel checks, as documented in Section 11 and the repository. AI systems are not authors and cannot accept responsibility for the work. Before submission or archival deposit, the author must inspect and approve the final manuscript, verify citations and originality, and accept full responsibility for its accuracy and integrity. The final archival version must replace this provisional statement with a complete retained-record inventory of the services, model or version identifiers when available, purposes, and affected research stages. No generative-AI-created figures are included.

Author contributions and approval

Dana Edwards is the named author. Affiliation, ORCID, corresponding-author contact, and a contribution statement remain pending. This review copy must not be deposited until the author approves the final text and accepts accountability for the work.

Data, code, and reproducibility

No empirical or personal data were used. Source code, proof artifacts, deterministic checkers, and frozen receipts are available in the public [OrbitSynthesis repository](#) and in [draft PR 16](#). Section 11 gives the exact reproduction map. The principal manuscript source hash is 01ddf4565212f798d0adf5c4c9b16d6de9074a317a34f2a39a80051de55a5410.

Funding and competing interests

Funding and competing-interest declarations have not yet been supplied by the author and must be completed before submission. No legal opinion about patents, freedom to operate, or license scope is made in this manuscript.

Ethics statement

This work is mathematical and computational and involved no human participants, animals, personal data, or clinical intervention.

References

1. A. F. Pixley, "Functionally Complete Algebras Generating Distributive and Permutable Classes," *Mathematische Zeitschrift* 114 (1970), 361–372. <https://eudml.org/doc/171324>
2. A. F. Pixley, "The Ternary Discriminator Function in Universal Algebra," *Mathematische Annalen* 191 (1971), 167–180. <https://eudml.org/doc/162131>

3. K. A. Baker and A. F. Pixley, "Polynomial Interpolation and the Chinese Remainder Theorem for Algebraic Systems," *Mathematische Zeitschrift* 143 (1975), 165–174. <https://eudml.org/doc/172222>
4. R. W. Quackenbusch, "Demi-Semi-Primal Algebras and Mal'cev-Type Conditions," *Mathematische Zeitschrift* 122 (1971), 166–176. <https://eudml.org/doc/171592>
5. C. E. Shannon, "The Synthesis of Two-Terminal Switching Circuits," *Bell System Technical Journal* 28 (1949). <https://www.nokia.com/bell-labs/publications-and-media/publications/the-synthesis-of-two-terminal-switching-circuits/>
6. O. B. Lupanov, "On the Principle of Local Coding and the Realization of Functions in a Certain Class of Networks Composed of Functional Elements," *Doklady Akademii Nauk SSSR* 140(2) (1961), 322–325. <https://www.mathnet.ru/eng/dan25510>
7. S. V. Yablonskii, "Functional Constructions in a k-Valued Logic," *Trudy Matematicheskogo Instituta imeni V. A. Steklova* 51 (1958), 5–142. <https://www.mathnet.ru/eng/tm1275>
8. V. A. Orlov, "Complexity of Implementing Functions of k-Valued Logic by Circuits and Formulas in Functionally Complete Bases," *Discrete Applied Mathematics* 135(1–3) (2004), 223–233. [https://doi.org/10.1016/S0166-218X\(02\)00306-2](https://doi.org/10.1016/S0166-218X(02)00306-2)
9. A. V. Kochergin, "On the Depth of k-Valued Logic Functions Over Arbitrary Bases," *Journal of Mathematical Sciences* 233(1) (2018), 100–102. <https://doi.org/10.1007/s10958-018-3927-5>
10. R. F. Safin, "On a Relation Between Depth and Complexity in Precomplete Classes of k-Valued Logic," *Mathematical Problems of Cybernetics* 13 (2004), 223–278 [Russian]. https://keldysh.ru/papers/2004/mvk/mvk2004_223.pdf
11. S. A. Lozhkin, "On Synthesis of Formulas Whose Complexity and Depth Do Not Exceed Asymptotically Best Estimates of High Accuracy," *Moscow University Mathematics Bulletin* 62(3) (2007), 93–100. <https://www.mathnet.ru/eng/vmumm1048>
12. S. A. Lozhkin, "On the Depth of a Multiplexer Function with a Small Number of Select Lines," *Mathematical Notes* 115(5) (2024), 748–754. <https://www.mathnet.ru/eng/mzmm14190>
13. A. D. Korshunov, "Computational Complexity of Boolean Functions," *Russian Mathematical Surveys* 67(1) (2012), 93–165. <https://www.mathnet.ru/eng/rm9459>
14. E. Lehtonen and A. Szendrei, "Equivalence of Operations with Respect to Discriminator Clones," *Discrete Mathematics* 309 (2009), 673–685. <https://arxiv.org/abs/0706.0195>
15. 吴训威 and 陈偕雄, "具有三轨输出的三值触发器及其在三值时序电路中的应用," *中国科学 A 辑* 1985(7), 643–653. <https://www.sciengine.com/doi/pdf/39cfd4607c82435d895d35165b3b28c0>
16. 徐明强, 管致锦, and 张海豹, "基于最小混乱度的三值可逆逻辑综合算法," *电子学报* 41(7) (2013), 1352–1357. <https://sns.wanfangdata.com.cn/sns/perio/dianzixb/?isSync=0&issueNum=07&page=2&publishYear=2013&tabId=article>
17. P. B. Tarasov, "Uniformity of Certain Systems of Functions of Many-Valued Logic," *Moscow University Mathematics Bulletin* 68(2) (2013), 99–102. <https://m.mathnet.ru/eng/vmumm398>
18. P. B. Tarasov, "Several Conditions for Uniformity of a Finite System of Many-Valued Logic," *Uchenye Zapiski Kazanskogo Universiteta. Seriya Fiziko-Matematicheskie Nauki* 156(3) (2014), 123–131. <https://www.mathnet.ru/eng/uzku1272>
19. N. Ishiura, "Synthesis of Multilevel Logic Circuits from Binary Decision Diagrams," *IEICE Transactions on Information* E76-D(9) (1993), 1085–1092. https://globals.ieice.org/en_transactions/information/10.1587/e76-d_9_1085/_p
20. J. Backes, *Algorithms and Data Structures for Logic Synthesis and Verification*, Ph.D. dissertation, University of Minnesota (2013), dual-rail ternary encoding discussion. <https://loonwerks.com/publications/pdf/backes2013phd.pdf>
21. L. G. Valiant, "Universal Circuits (Preliminary Report)," *Proceedings of STOC 1976*, 196–203. <https://doi.org/10.1145/800113.803649>

22. S. A. Cook and H. J. Hoover, “A Depth-Universal Circuit,” *SIAM Journal on Computing* 14(4) (1985), 833–839. <https://doi.org/10.1137/0214058>
- 1015 23. J. Holmgren and R. Rothblum, “Linear-Size Boolean Circuits for Multiselection,” *CCC 2024*, 11:1–11:20. <https://doi.org/10.4230/LIPIcs.CCC.2024.11>
24. W. F. McColl and M. S. Paterson, “The Depth of All Boolean Functions,” *SIAM Journal on Computing* 6(2) (1977), 373–380. <https://doi.org/10.1137/0206026>
- 1020 25. 张会红, 陈治文, and 汪鹏君, “二叉决策图映射电路的面积和延时优化,” *电子与信息学报* 41(3) (2019), 725–731. <https://doi.org/10.11999/JEIT180443>
26. 何新华, 宫云战, and 魏道政, “面向多输出电路的 BDD 拼接构造,” *电子与信息学报* 19(3) (1997), 356–360. <https://jeit.ac.cn/article/id/85665992-ae08-49a4-83ca-1381a6d4735f>
27. V. A. Orlov, “Realization of k-Valued Functions by Circuits of Functional Elements,” *Mathematical Notes* 64 (1998), 371–376. <https://doi.org/10.1007/BF02314847>
- 1025 28. S. B. Gashkov, “On the Depth of Boolean Functions,” *Problemy Kibernetiki* 34 (1978), 265–268 [Russian]. [https://publ.lib.ru/ARCHIVES/P/%27%27Problemy_kibernetiki%27%27_%28seriya%29/%cf%f0%ee%e1%eb%e5%ec%fb%20%ea%e8%e1%e5%f0%ed%e5%f2%e8%ea%e8.%20%c2%fb%ef%f3%f1%ea%2034.\(1978\).pdf](https://publ.lib.ru/ARCHIVES/P/%27%27Problemy_kibernetiki%27%27_%28seriya%29/%cf%f0%ee%e1%eb%e5%ec%fb%20%ea%e8%e1%e5%f0%ed%e5%f2%e8%ea%e8.%20%c2%fb%ef%f3%f1%ea%2034.(1978).pdf)
- 1030 29. R. E. Ladner and M. J. Fischer, “Parallel Prefix Computation,” *Journal of the ACM* 27(4) (1980), 831–838. <https://doi.org/10.1145/322217.322232>
30. R. P. Brent and H. T. Kung, “A Regular Layout for Parallel Adders,” *IEEE Transactions on Computers* C-31(3) (1982), 260–264. <https://doi.org/10.1109/TC.1982.1675982>